

Cardiff Metropolitan University

in collaboration with ICBT Campus

BSc (Hons) in Software Engineering

CIS6003 - Advanced Programming

Individual Coursework (WRIT1)

Sunrise Dental Clinic

Appointment & Patient Management System

Task A - System Design (UML)

STUDENT NAME: [Your Full Name]

STUDENT ID: [GL/BSCSD/xx/xx]

CARDIFF USER ID: [stxxxxxxxx]

Cardiff Metropolitan University

BSc (Hons) in Software Engineering

Assignment Cover Sheet

Name	[Your Full Name]
Student ID	[GL/BSCSD/xx/xx] - Cardiff User ID: [stxxxxxxxx]
Unit code	CIS6003
Unit title	Advanced Programming
Year / Study period	Year 3 / [Study Period]
Lecturer	[Lecturer Name]
Mode of delivery	Full Time
Nature of the assessment	Individual Coursework (WRIT1)
Topic of the case study	Sunrise Dental Clinic - Appointment & Patient Management System
Word count	[to be completed]
Due date / Time	[Due Date]

Declaration

I certify that the attached material is my original work. No other person's work or ideas have been used without acknowledgement. Except where I have clearly stated that I have used some of this material elsewhere, I have not presented it for examination / assessment in any other course or unit at this or any other institution.

Name / Signature:	Date:
--------------------------	--------------

Acknowledgement

I would like to take this opportunity to thank everyone who supported me while I was completing this Advanced Programming coursework. My sincere thanks go to my lecturer at ICBT Campus for the guidance, feedback and encouragement given throughout the module, which helped me understand how the theory of object-oriented design and design patterns is actually applied when building a real system.

I am also grateful to my family and my colleagues for their patience and support during the busy weeks of this project. Finally, I would like to thank the staff of Sunrise Dental Clinic whose day-to-day way of working inspired the requirements behind this system. Any remaining mistakes in this report are my own.

Table of Contents

Table of Figures

Introduction

Sunrise Dental Clinic is a growing dental practice that currently manages its patients, dentist availability, appointments and billing mostly on paper and spreadsheets. As the number of patients has increased, this manual approach has started to cause problems such as double-booked slots, lost patient records and slow, error-prone billing. The purpose of this project is to design and build an Appointment & Patient Management System that removes these problems and lets patients, receptionists, dentists and administrators work from one shared, reliable source of information.

The proposed system is a web application. Patients use a self-service portal to register, view dentist availability, book or cancel appointments and download their bills, while clinic staff use a back-office area to manage slots, register walk-in appointments, keep patient records and generate bills. The backend is implemented in Java with the Spring Boot framework, the frontend is a React single-page application, user identity is handled by Firebase Authentication, and data is stored in Cloud Firestore. The design also makes deliberate use of well-known design patterns (Singleton, Factory, Observer and Strategy) so that the code stays clean, testable and easy to extend.

This document covers Task A of the coursework: the object-oriented design of the system expressed through UML. It presents and explains the use case diagram, the domain class diagram and a sequence diagram for the core “Book Appointment” workflow. For each diagram I first introduce its purpose, then give an overview in the context of Sunrise Dental Clinic, explain the important design decisions and relationships, and close with a short conclusion. The later tasks (implementation, testing and version control / CI-CD) build directly on the design described here.

Task A - System Design (UML)

Task A defines the shape of the system before any code is written. Three UML views are used because each answers a different question: the use case diagram shows who uses the system and what they can do, the class diagram shows how the information in the system is structured, and the sequence diagram shows how the parts of the system talk to each other over time to complete an important job. Together they give a clear and consistent picture that the implementation in the following tasks can follow.

Use Case Diagram

Introduction to the Use Case Diagram

A use case diagram describes the functional requirements of a system from the point of view of the people (and other systems) that interact with it. It does not show how anything works internally; it simply captures the goals that each actor wants to achieve. For Sunrise Dental Clinic this is a useful starting point because the clinic has several very different types of user, and being clear about who is allowed to do what is important both for usability and for security.

Because the system has a large number of use cases, I have separated them into two diagrams so that each one stays clean and easy to read. The first diagram covers the patient self-service portal, and the second covers the staff back-office. This mirrors how the real application is split and keeps the relationships between actors and use cases uncluttered.

Overview of the Sunrise Dental Clinic System

Four human actors interact with the system. The Patient books and manages their own appointments and bills. The Receptionist runs the front desk: managing dentist availability, registering walk-in or phone appointments, keeping patient records and producing bills. The Dentist views their daily schedule and marks treatments as completed. The Administrator has the widest access and manages staff accounts, the treatment catalogue and pricing, and clinic reports. In addition, two secondary (system) actors are shown: Firebase Authentication, which verifies user credentials, and the Email / SMS Gateway, which delivers confirmation messages to patients.

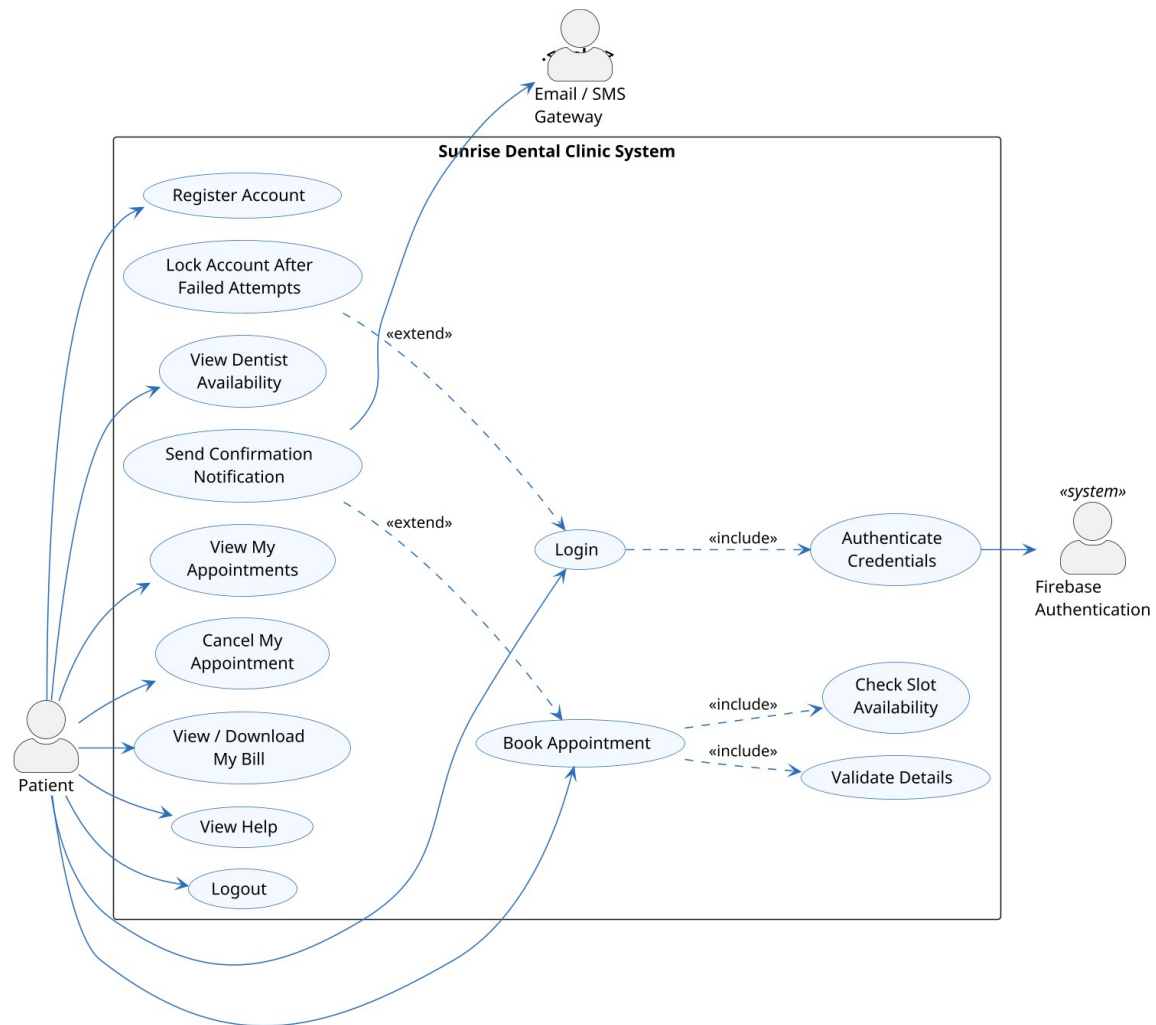


Figure 1 - Use Case Diagram (1 of 2) - Patient Self-Service Portal

The patient portal groups all of the self-service goals a patient can reach on their own: registering an account, logging in, viewing dentist availability, booking and cancelling appointments, viewing or downloading a bill, and getting help. Booking an appointment is the most important patient goal and it is deliberately broken down into smaller included steps, which is discussed below.

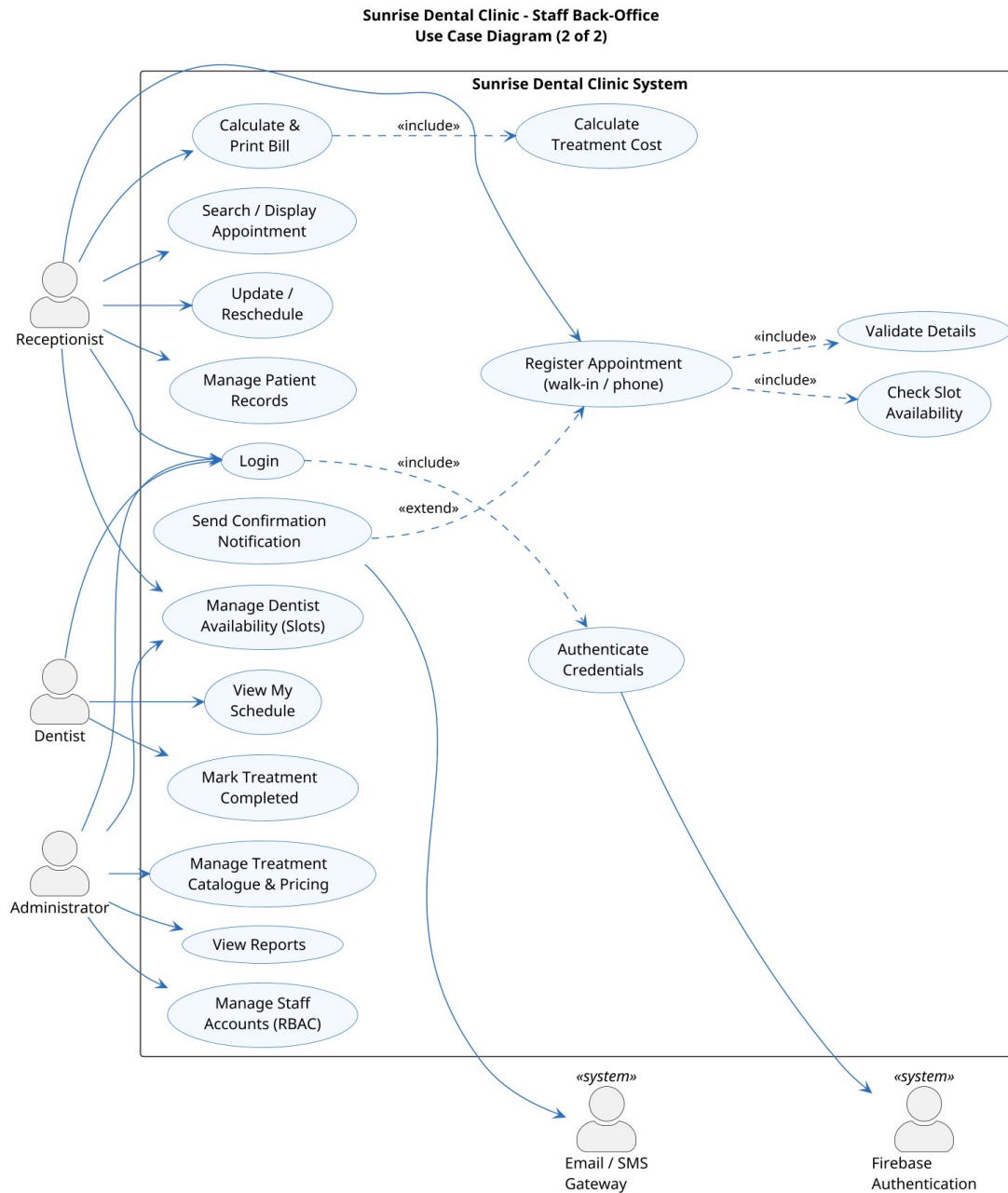


Figure 2 - Use Case Diagram (2 of 2) - Staff Back-Office

The staff diagram shows the receptionist, dentist and administrator sharing the same login but each reaching a different set of use cases according to their role. The administrator is intentionally shown with a superset of privileges (for example, managing dentist availability as well as staff accounts and reports), which reflects the role-based access control that the system enforces.

Design Decisions and Relationships

Beyond the plain actor-to-use-case associations, the diagrams use include and extend relationships to show shared and optional behaviour, which keeps the model accurate and avoids repetition:

- Login «include» Authenticate Credentials – every login must verify the user, and that check is delegated to Firebase Authentication. Modelling it as an included use case shows it is a mandatory sub-step that is reused wherever a user signs in.
- Book Appointment / Register Appointment «include» Validate Details and Check Slot Availability – both the patient booking and the receptionist walk-in flows always validate the request and confirm the chosen slot is still free, so these steps are shared through include relationships rather than being duplicated.
- Calculate & Print Bill «include» Calculate Treatment Cost – producing a bill always requires the treatment cost to be worked out first.
- Send Confirmation Notification «extend» Book / Register Appointment – sending an e-mail or SMS confirmation is useful but not essential to completing the booking, so it is modelled as an extension. If the gateway is unavailable the appointment is still created.
- Lock Account After Failed Attempts «extend» Login – account lock-out is optional behaviour that only occurs on the exceptional path of repeated failed logins, which is exactly what extend is meant to express.

Separating the model into a patient view and a staff view, and using include for mandatory shared steps and extend for optional or exceptional behaviour, keeps each requirement in exactly one place. This makes the diagrams easier to read and gives a dependable checklist of functionality for the later tasks.

Conclusion

The two use case diagrams capture the full set of functional requirements for Sunrise Dental Clinic and make the boundaries between the four roles explicit. They confirm that the system supports patients helping themselves online while giving staff the tools they need at the front desk, and they set up the include and extend relationships that the class and sequence designs then realise.

Class Diagram

Introduction to the Class Diagram

The class diagram describes the static structure of the system: the classes that hold the information, their attributes and operations, and the relationships between them. Where the use case diagram answered “what can the system do?”, the class diagram answers “what does the system need to remember, and how is that information connected?”. It is the blueprint that the Java domain model is built from.

Overview of the Sunrise Dental Clinic System

The model is organised around a few clear areas: identity and people (UserAccount, Patient, Dentist), scheduling (DentistSession and Slot), the treatment catalogue (Treatment), and the core clinical and financial records (Appointment, Bill, Notification). Supporting classes such as AuditEvent and AiSettings, and a set of enumerations (Role, AppointmentStatus, SlotStatus, ChannelType and NotificationStatus), keep status values and configuration type-safe rather than relying on loose strings. Access modifiers are shown so that attributes are kept private (encapsulation) and are reached only through public operations.

Sunrise Dental Clinic - Domain Class Diagram
(access modifiers, multiplicity, aggregation, composition)

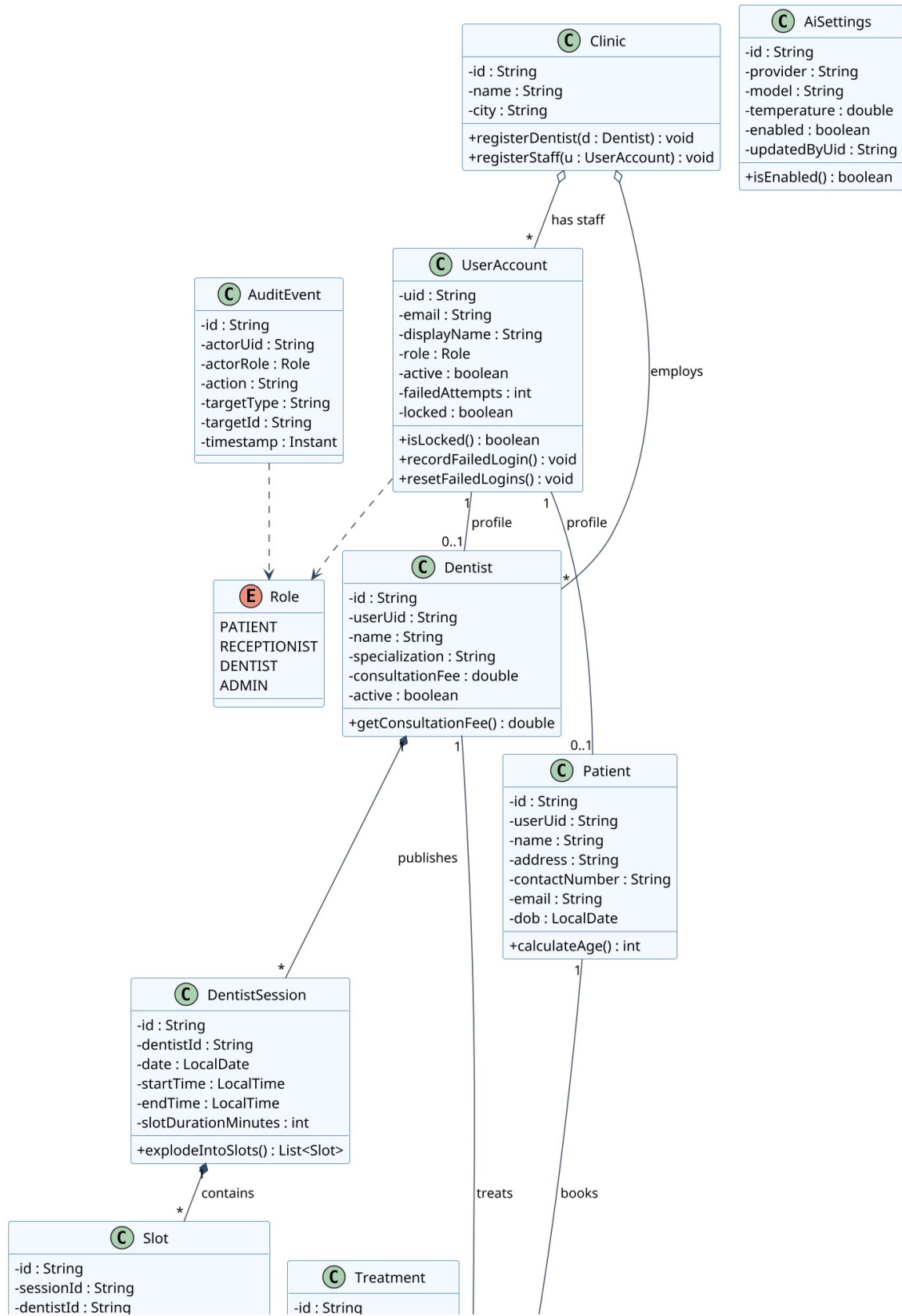


Figure 3 - Domain Class Diagram - Sunrise Dental Clinic

Key Classes and Their Relationships

A few classes and relationships carry most of the meaning of the model:

- UserAccount is the single identity record and holds the user's role and the login-security fields (failedAttempts, locked). A UserAccount optionally has a Patient or Dentist profile, which cleanly separates “who can log in” from “who they are in the clinic”.
- A Dentist publishes DentistSession objects, and each session explodes into a number of fixed-length Slot objects. This is modelled as composition because a slot has no meaning without the session that created it – if the session is removed, its slots go with it.
- Appointment is the central class of the whole system. It ties together a patient, a dentist, a slot and a treatment, and it carries its own status (CONFIRMED, COMPLETED, BILLED, CANCELLED). A Bill and any Notification records are composed inside the appointment, because they cannot exist without it.
- Clinic aggregates its Dentists and staff UserAccounts. Aggregation (rather than composition) is used here on purpose, because a dentist or a member of staff is a person who continues to exist even if the clinic record is removed.

Multiplicities are shown on every association to make the rules precise: one dentist has many sessions, one session has many slots, one slot reserves at most one appointment (0..1), and one patient may have many appointments over time.

Design Decisions and Relationships

The main design decisions in the class diagram are about choosing the right kind of relationship and keeping the classes cohesive:

- Composition vs aggregation – parts that die with their whole (Slot within DentistSession; Bill and Notification within Appointment) use composition, while parts that live independently (Dentist and UserAccount within Clinic) use aggregation. Choosing correctly here documents the real lifecycle rules of the data.
- Encapsulation and behaviour – instead of being plain data holders, classes own the behaviour that belongs to them: Slot.book() and isBookable() protect the booking rule,

`Appointment.markCompleted()` and `cancel()` guard valid status changes, and `DentistSession.explodeIntoSlots()` creates the bookable slots. This keeps logic close to the data it acts on.

- Enumerations for status and type – using enums such as `AppointmentStatus` and `SlotStatus` instead of free text prevents invalid values and makes the state of an appointment or slot unambiguous.
- Confidentiality – the diagnosis field on `Appointment` is marked as confidential (dentist and patient only), recording an access rule directly in the design so it is not forgotten during implementation.

Conclusion

The class diagram gives a well-structured, encapsulated domain model in which `Appointment` sits at the centre and every relationship reflects a real rule of the clinic. The careful use of composition, aggregation, multiplicity and enumerations means the model is both accurate and ready to be turned directly into the Java classes used in Task B.

Sequence Diagram

Introduction to the Sequence Diagram

A sequence diagram shows how objects collaborate to carry out one particular scenario, with time running from top to bottom and messages passing between the participants. While the class diagram shows the structure and the use case diagram shows the goals, the sequence diagram shows the actual order of interactions needed to achieve a goal. I have chosen the “Book Appointment” scenario because it is the most important workflow in the system and it exercises several of the design patterns used in the project.

Overview of the Book Appointment Workflow

The scenario begins when a patient chooses an open slot and a treatment and confirms the booking in the React portal. The request travels to the `AppointmentController`, which validates it and passes it to the `AppointmentService`. The service performs the booking inside a single atomic transaction so that two patients can never book the same slot at the same time. Once the

appointment is safely stored, an event is published and a confirmation notification is sent, and finally the new appointment number is returned to the patient.

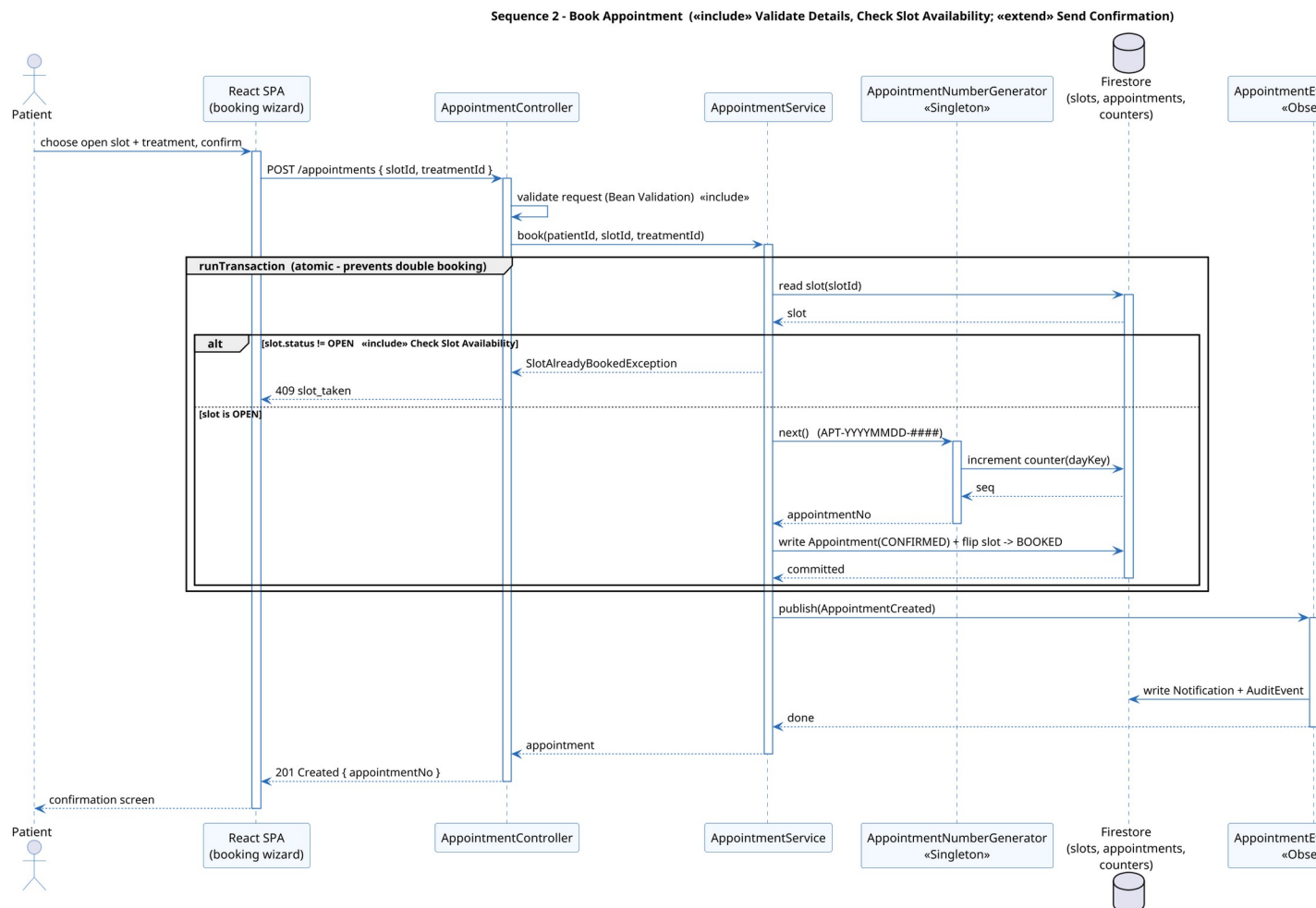


Figure 4 - Sequence Diagram - Book Appointment

Design Decisions and Relationships in the Sequence Diagram

The diagram highlights several deliberate design decisions, many of which correspond to the include and extend relationships from the use case model and to specific design patterns:

- Atomic slot booking – reading the slot, generating the appointment number and flipping the slot to BOOKED all happen inside one transaction. The alt fragment shows the two possible outcomes: if the slot is no longer OPEN the service returns a 409 “slot taken” error, otherwise the booking is committed. This is how the design prevents double-booking, which was one of the clinic’s main problems.

- Singleton pattern – the AppointmentNumberGenerator is a Singleton that produces unique, sequential appointment numbers in the form APT-YYYYMMDD-#### by safely incrementing a daily counter.
- Observer pattern – after the appointment is created the service simply publishes an AppointmentCreated event through the AppointmentEventPublisher. Observers react to it without the service needing to know about them, which keeps the booking logic decoupled from side effects such as notifying the patient and writing the audit record.
- Factory pattern and the extend relationship – sending the confirmation corresponds to the “Send Confirmation Notification” extension. The correct channel object (here an EmailChannel) is produced by a factory, so the appointment is still created successfully even if the notification step fails.
- Included validation – the “validate request” self-call and the slot-availability check in the diagram are the sequence-level realisation of the Validate Details and Check Slot Availability include use cases, showing that the design stays consistent across the different UML views.

Conclusion

The Book Appointment sequence diagram shows exactly how the layers of the system – React portal, controller, service, patterns and database – cooperate to complete the clinic’s most important task safely. It demonstrates that the design not only lists the right features but also handles the difficult parts, such as preventing double-booking and keeping side effects decoupled, in a clean and reliable way. The same style of interaction is reused for the other workflows (login, billing and AI symptom triage) that are detailed in the later tasks.